

Software Design Document

SPI Driver Module

Lab 05 – Microcontrollers and Embedded Systems

Module	SPI Driver
Version	1.0.0
Author(s)	J.M. Páez Sandoval, L.A. Lugo Muñiz, D. Lopez Moreno, C.A. Martínez Macías
Institution	CETYS Universidad — Facultad de Ingeniería
Date	29/05/2026

1. Introduction

1.1 Purpose

This document describes the design and architecture of the `spi_driver` module for Lab 05. It covers the GPIO and SPI1 peripheral initialization sequence, polled transmit/receive operations, chip-select control, and the full-duplex dummy-byte receive pattern.

1.2 Scope

The `spi_driver` is a low-level bare-metal module for the STM32F411RE SPI1 peripheral. It operates in master mode using polling (no interrupts, no DMA). The fixed configuration is SPI Mode 3 (CPOL=1, CPHA=1), 8-bit frames, MSB-first, software NSS, $f_{PCLK}/32 \approx 3.1$ MHz. GPIO configuration is handled internally via `gpio_driver`. The module exposes five public functions and four private helpers that encapsulate the four initialization phases.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SPI	Serial Peripheral Interface — synchronous four-wire full-duplex bus
MOSI	Master Out Slave In — data driven by master to slave
MISO	Master In Slave Out — data driven by slave to master
SCK	Serial Clock — generated by master; controls data sampling
CS / NSS	Chip Select / Slave Select — active-low, selects the slave device
CPOL	Clock Polarity — 0=SCK idles LOW, 1=SCK idles HIGH
CPHA	Clock Phase — 0=data captured on first edge, 1=on second edge

Term	Definition
Mode 3	CPOL=1, CPHA=1 — SCK idles HIGH, data captured on rising edge
MSB first	Most Significant Bit transmitted first (LSBFIRST=0)
TXE	TX buffer Empty flag (SR bit 1) — safe to write next byte
RXNE	RX buffer Not Empty flag (SR bit 0) — received byte available
BSY	Busy flag (SR bit 7) — SPI peripheral actively transferring
OVR	Overrun error — new byte received before previous DR was read
SSM / SSI	Software Slave Management / Internal select bit (master mode)
BR[2:0]	Baud Rate prescaler bits [5:3] in CR1 — fPCLK divisor
DFF	Data Frame Format — 0=8-bit, 1=16-bit
SPE	SPI Enable bit (CR1 bit 6) — activates the peripheral
SPI_Status_t	Return type: SPI_OK=0, SPI_ERROR=1

1.4 Requirements

All requirements are allocated in Docs/SRS/SPI_Driver_SRS.md. Functional requirements FR-1 to FR-8 and non-functional requirements NFR-1 to NFR-3 are addressed.

2. System Architecture

2.1 High-Level Design

The spi_driver sits between the application layer and the SPI1 hardware registers. GPIO initialization is delegated to gpio_driver. Four private helpers encapsulate the four initialization phases; only five public functions are exposed to callers.

Layer	Modules
Application	main.c — calls spi_cs_enable(), spi_transmit(), spi_receive(), spi_cs_disable()
SPI Driver	GPIO config (via gpio_driver) + SPI1 peripheral config + polled TX/RX
GPIO Driver	gpio_driver.c — pin mode, AF5, speed, push-pull configuration
Hardware	STM32F411RE SPI1 (APB2). PA5=SCK, PA6=MISO, PA7=MOSI (AF5). PA9=CS (output).

2.2 Module Interfaces

Public API declared in spi_driver.h. Dependencies: stm32f411xe.h, gpio_driver.h, stdint.h. No HAL. Hardware pins are fixed: PA5=SCK, PA6=MISO, PA7=MOSI (AF5), PA9=CS (push-pull output) on SPI1.

3. Detailed Design

3.1 Class Diagram

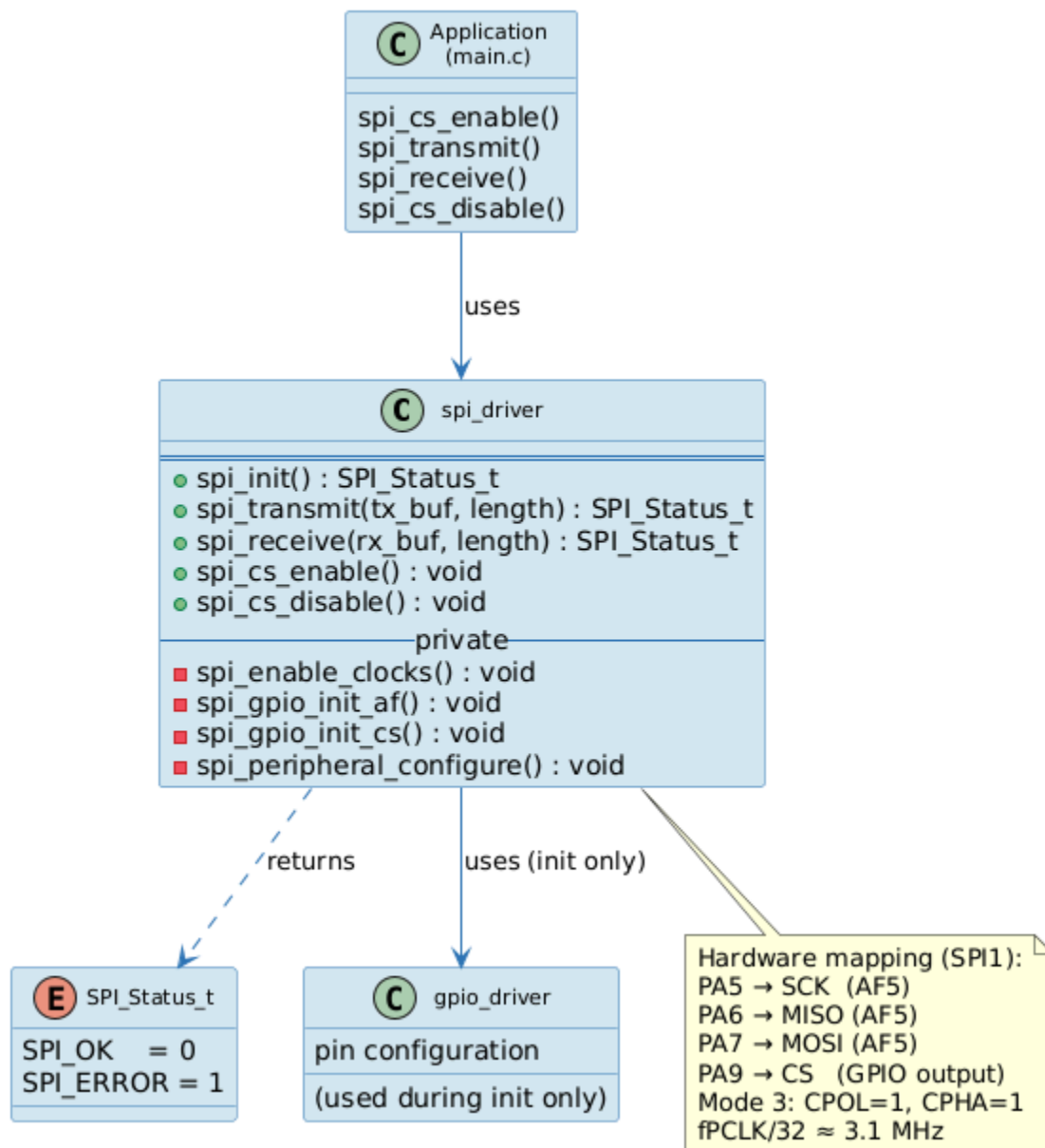


Figure 1. Class Diagram — `spi_driver` module, `SPI_Status_t`, `gpio_driver` dependency, hardware mapping

3.2 Module Behavior

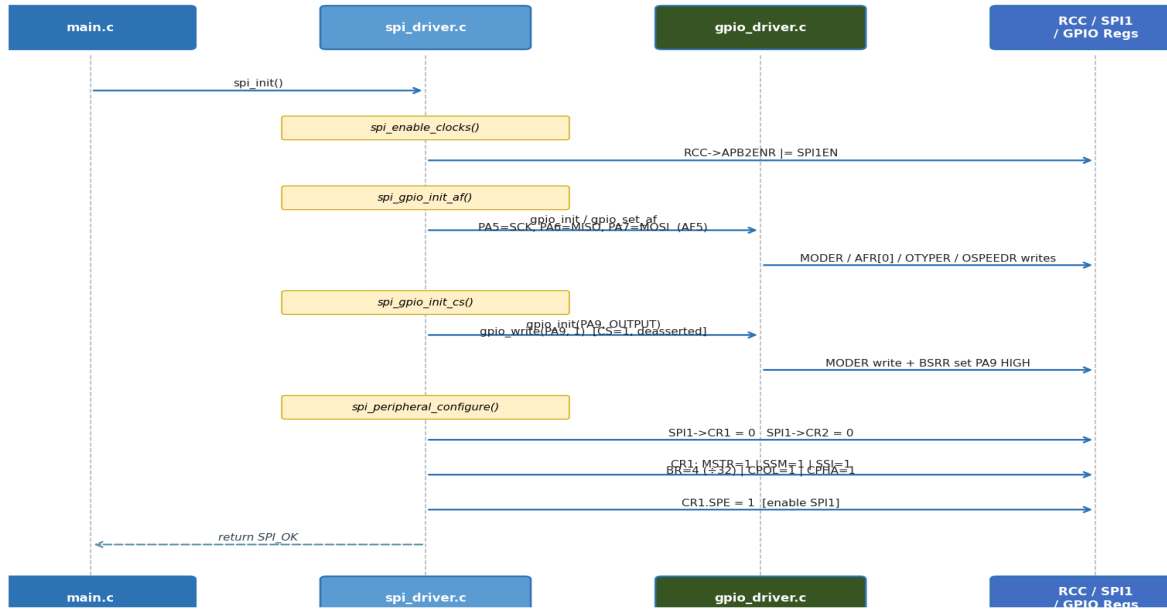


Figure 2. Sequence Diagram — `spi_init()`: clock enable, GPIO AF config, CS init, CR1 write, `SPE=1`

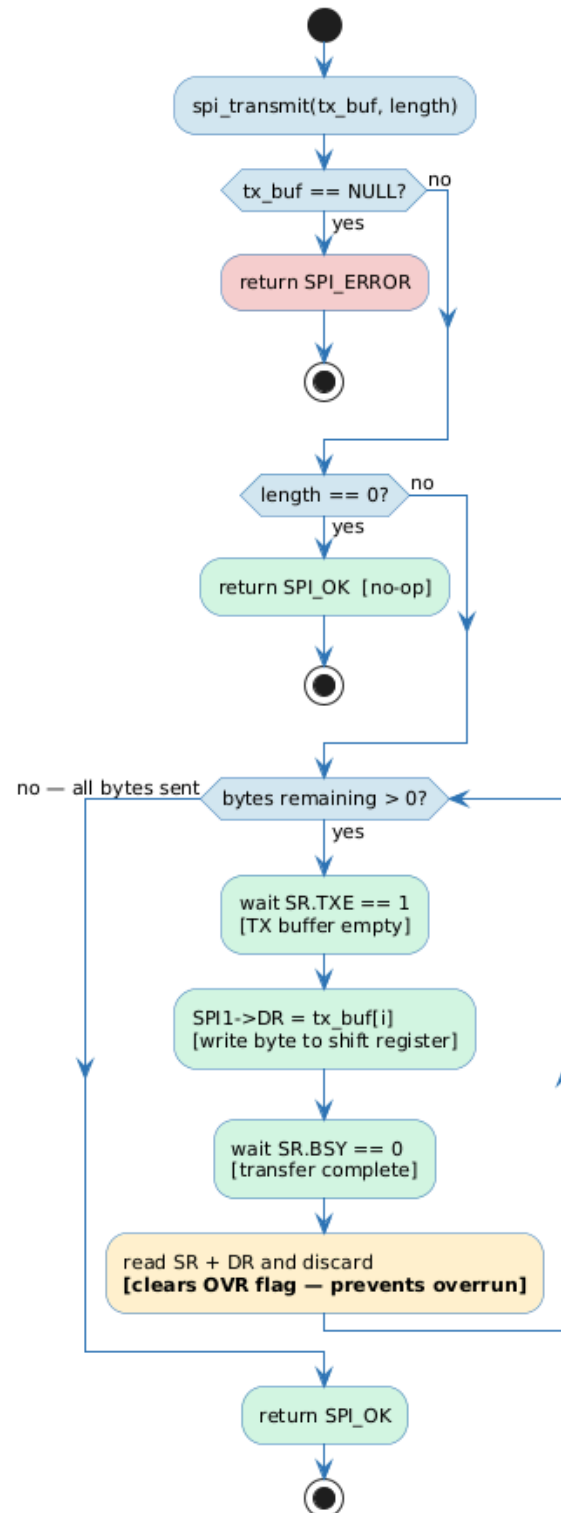


Figure 3. Activity Diagram — `spi_transmit()`: TXE→DR write→BSY wait→DR discard per byte

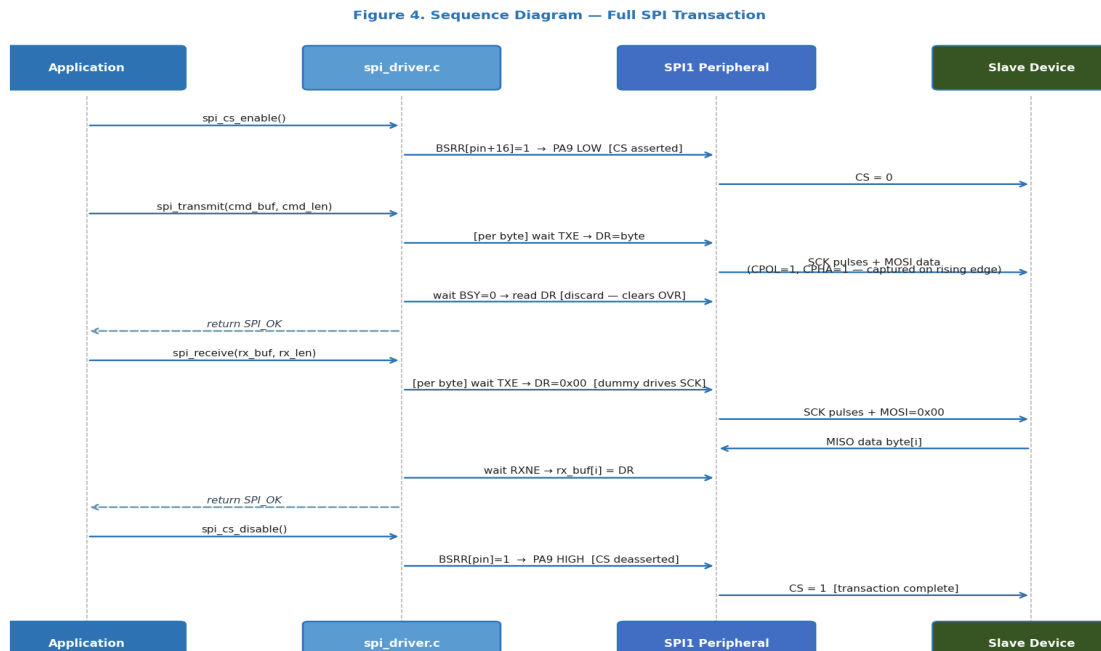


Figure 4. Sequence Diagram — Full transaction: `cs_enable` → `transmit` → `receive` → `cs_disable`

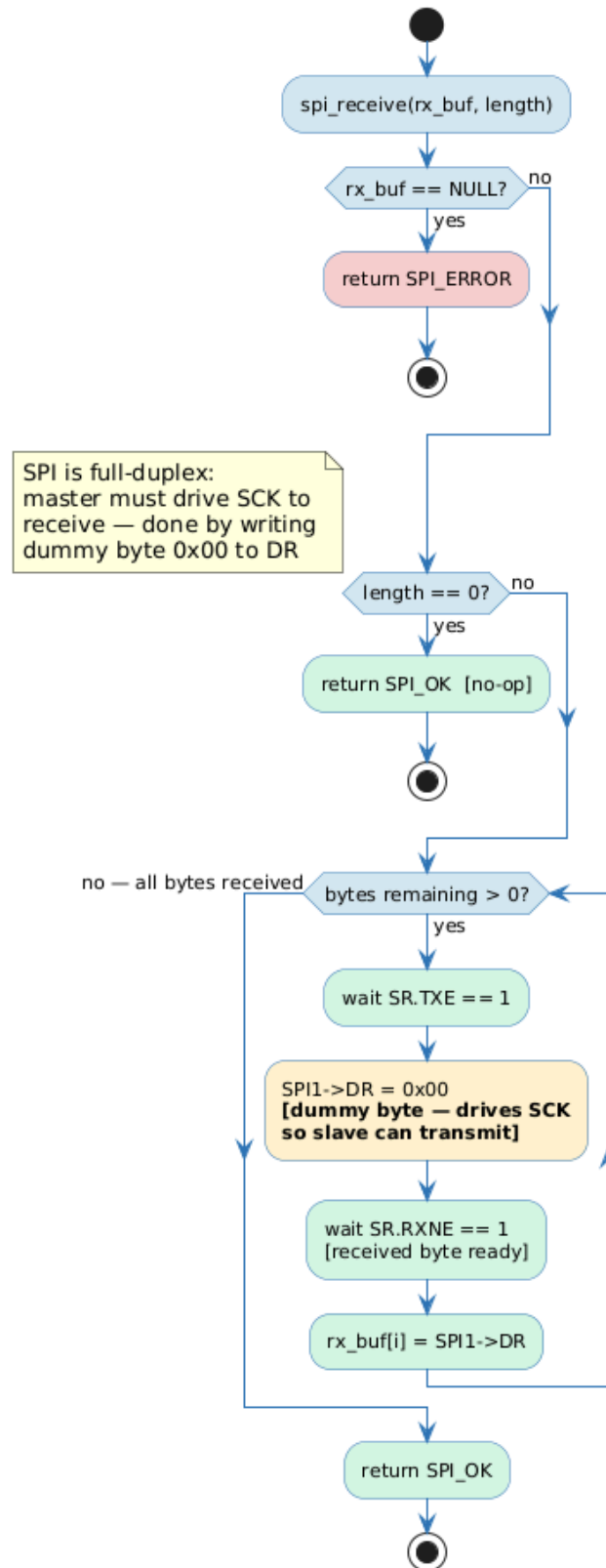


Figure 5. Activity Diagram — `spi_receive()`: dummy byte drives SCK, `RXNE`→`DR` read per byte

Figure 6. State Diagram — SPI States

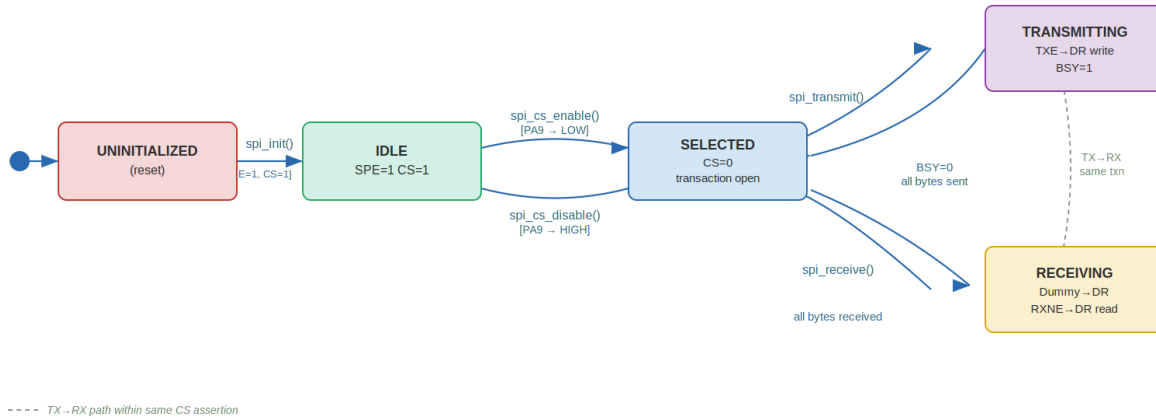


Figure 6. State Diagram — SPI States

3.3 SPI Driver Module Description

`spi_init()` calls four private helpers in sequence. `spi_enable_clocks()` sets `RCC->APB2ENR.SPI1EN`. `spi_gpio_init_af()` configures PA5, PA6, PA7 as AF5 push-pull very-high-speed via `gpio_driver`. `spi_gpio_init_cs()` configures PA9 as push-pull output and drives it HIGH (deasserted). `spi_peripheral_configure()` clears CR1/CR2, writes `MSTR=1`, `SSM=1`, `SSI=1`, `BR=4` ($\div 32$), `CPOL=1`, `CPHA=1`, and finally sets `SPE=1` last to ensure all parameters are stable.

`spi_transmit()` iterates over the TX buffer. Per byte: waits for TXE, writes DR, waits for BSY=0, then reads and discards DR. The discard step is critical — without it, the OVR (overflow) flag accumulates and corrupts subsequent RXNE-based reads.

`spi_receive()` uses the full-duplex nature of SPI: the master must generate SCK to clock in data from the slave. Per byte: waits for TXE, writes dummy 0x00 (drives SCK), waits for RXNE, reads DR. The slave transmits its data synchronized to the SCK pulses generated by the dummy write.

`spi_cs_enable()` uses `gpio_write(GPIOA, 9, 0) → BSRR` atomic reset (PA9 LOW). `spi_cs_disable()` uses `gpio_write(GPIOA, 9, 1) → BSRR` atomic set. The application controls CS framing around each transaction.

4. Application Programming Interfaces

4.1 Data Types and Macros

Type / Macro	Values	Notes
SPI_Status_t	SPI_OK=0SPI_ERROR=1	Return type for init, transmit, receive
SPI_PORT	GPIOA	Shared port for SCK/MISO/MOSI/CS
SPI_PIN_SCK	5U	PA5 — Serial Clock (AF5)
SPI_PIN_MISO	6U	PA6 — Master In Slave Out (AF5)
SPI_PIN_MOSI	7U	PA7 — Master Out Slave In (AF5)
SPI_PIN_CS	9U	PA9 — Chip Select (GPIO output, active-low)
SPI_BR_DIV32	4U	BR[2:0]=4 → fPCLK/32 ≈ 3.1 MHz

4.2 Error Codes

Code	Value	Definition
SPI_OK	0	Operation completed successfully
SPI_ERROR	1	NULL pointer passed as tx_buf or rx_buf

4.3 Function Definitions

spi_init

Function Name	spi_init
Syntax	SPI_Status_t spi_init(void)
Parameters (in)	None
Return value	SPI_OK (hardware assumed present)
Description	Enables APB2 clock, configures PA5/PA6/PA7 as AF5 push-pull VHIGH, PA9 as output CS=1, writes CR1 Mode3/Master/SSM/BR÷32, sets SPE=1. Satisfies FR-1, FR-2.
Constraints	Must be called before any other spi_* function.
Available via	spi_driver.h

spi_transmit

Function Name	spi_transmit
Syntax	SPI_Status_t spi_transmit(const uint8_t *tx_buf, uint16_t length)
Parameters (in)	tx_buf — source buffer (not NULL)length — number of bytes
Return value	SPI_OK or SPI_ERROR (tx_buf==NULL)
Description	Per byte: wait TXE → write DR → wait BSY=0 → discard DR (prevents OVR). Satisfies FR-3, FR-4.

Constraints	length=0 is a no-op. CS must be asserted by caller first.
Available via	spi_driver.h

spi_receive

Function Name	spi_receive
Syntax	SPI_Status_t spi_receive(uint8_t *rx_buf, uint16_t length)
Parameters (in)	rx_buf — destination buffer (not NULL)length — number of bytes
Return value	SPI_OK or SPI_ERROR (rx_buf==NULL)
Description	Per byte: wait TXE → DR=0x00 (dummy drives SCK) → wait RXNE → rx_buf[i]=DR. Satisfies FR-5, FR-6.
Constraints	SPI is full-duplex — receiving requires clock from dummy TX.
Available via	spi_driver.h

spi_cs_enable

Function Name	spi_cs_enable
Syntax	void spi_cs_enable(void)
Parameters (in)	None
Return value	void
Description	Drives PA9 LOW via BSRR atomic reset. Call before the first byte of a transaction. Satisfies FR-7.
Constraints	gpio_write(GPIOA, 9, 0).
Available via	spi_driver.h

spi_cs_disable

Function Name	spi_cs_disable
Syntax	void spi_cs_disable(void)
Parameters (in)	None
Return value	void
Description	Drives PA9 HIGH via BSRR atomic set. Call after the last byte of a transaction. Satisfies FR-8.
Constraints	gpio_write(GPIOA, 9, 1).
Available via	spi_driver.h

4.4 Internal Function Definitions

Private Function	Description
spi_enable_clocks()	RCC->APB2ENR = SPI1EN. Called once at start of spi_init().

Private Function	Description
<code>spi_gpio_init_af()</code>	PA5/PA6/PA7 → AF5, push-pull, VHIGH, no pull via <code>gpio_driver</code> .
<code>spi_gpio_init_cs()</code>	PA9 → output push-pull, drives HIGH (CS deasserted). Called by <code>spi_init()</code> .
<code>spi_peripheral_configure()</code>	Clears CR1/CR2, writes all SPI1 config bits, sets SPE=1 last.

5. Future Enhancements

- Add DMA-based `spi_transmit_dma()` / `spi_receive_dma()` to eliminate CPU busy-wait.
- Add 16-bit frame mode (DFF=1) for display controllers using 16-bit commands.
- Add `spi_transceive()` for simultaneous TX+RX in one call.
- Make pin/port assignments configurable via `spi_config_t` to support SPI2/SPI3.